PYQs

31 October 2026 15:06

QP Bundle 31

26 Oct 25

26 October 2025

One paper

```
1  def analyze(arr):
2      n = len(arr) 1
3      i = 1 1
4      while i < n:
5          print(arr[i])
6          i *= 2
```

What is the time complexity of the given function analyze?

i=1=2^0
i=2=2^1
i=4=2^2
i=2^k

k+2 lines of code

$2^k \le n$

$\log_2(2^k) \le \log_2(n)$

$k \le \log_2(n)$

$O(2+\log_2(n)\times 3)$
$= O(\log(n))$

OPTIONS :

- ○ $O(n)$
- ✓ $O(\log n)$
- ○ $O(\sqrt{n})$
- ○ $O(n \log n)$

$f_1(n) = n^2.(\log n)$

$f_2(n) = n.(\log n)^2$ $n(\log n)^2$

$f_3(n) = 2^n$

$f_4(n) = n^{1.5} = n.\sqrt{n}$ $f_5(n), f_2(n), f_4(n), f_1(n), f_3(n)$

$f_5(n) = (\log n)^{10}$ slowest growing

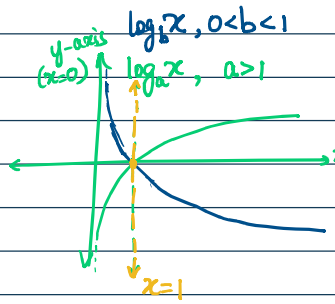
Arrange the above functions in increasing order of asymptotic complexity.

$\sqrt{1000000000} = 10000$

$(\log_{10}(100000000))^2 = 8^2 = 64$

OPTIONS :

- ○ $f_3(n), f_4(n), f_2(n), f_1(n), f_5(n)$
- ✓ $f_5(n), f_2(n), f_4(n), f_1(n), f_3(n)$
- ○ $f_4(n), f_3(n), f_2(n), f_1(n), f_5(n)$
- ○ $f_1(n), f_4(n), f_2(n), f_5(n), f_3(n)$



$\log_q p = r \equiv q^r = p$

$1^r = 1$ so $q=1$ useless

Base of logarithm $(0,1) \cup (1,\infty)$
OR $(0,\infty) - \{1\}$

$p > 0$

Given the sorted list:

`[3, 7, 11, 15, 19, 23, 27, 31, 35, 39, 43, 47, 51, 55, 59, 63, 67]`

If **binary search** is used to find the element `47`, which of the following shows the **correct sequence of comparisons**?

Note: Assume binary search computes the midpoint using: **(first index + last index) // 2**

fi = first index
li = last index

OPTIONS :

- ○ 35, 51, 39, 47
- ○ 35, 51, 47
- ✓ 35, 51, 43, 47
- ○ 35, 55, 39, 47
- ○ None of these

iteration 1 (0+16)//2 = 8

35 → search right → fi = 8+1, li = 16

iteration 2 (9+16)//2 = 12

51 → search left → fi = 9, li = 12−1

iteration 3 (9+11)//2 = 10

43 → search right → fi = 11, li = 11

iteration 4 (11+11)//2 = 11

47 return True
and also return index value = 11

Consider the following implementation of insertion sort.

```
1  def insertionsort(L):
2      n = len(L)
3      if n < 1:
4          return(L)
5      for i in range(n):
6          j = i
7          while(j > 0 and L[j] < L[j-1]):
8              (L[j],L[j-1]) = (L[j-1],L[j]) #Swap Operation
9              j = j-1
10     return(L)
```

The number of **swap** operations for `insertionsort([3, 4, 2, 1, 5])` is __.

≡ no. of times while loop is iterated

OPTIONS :

- ○ 2
- ○ 3
- ○ 4
- ✓ ○ 5

3rd iteration and 4th iteration
i=2 i=3
2 + 3

Total 5 swaps

i=0, j=0 no swap, while condtn evaluated to false

i=1, j=1 no swap, "

i=2, j=2

[3,2,4,1,5]
j=1
[2,3,4,1,5]
j=0
-- 2 swap -- ≡ 2 iterations

i=3, j=3

[2,3,1,4,5]
j=2
[2,1,3,4,5]
j=1
[1,2,3,4,5]
j=0
-- 3 swap -- ≡ 3 iteration of while loop

i=4, j=4 no swap
(no while loop iteration)

A list of `n` strings, each of length `k`, is sorted using **Merge Sort** in lexicographical order. Assume that comparing two strings of length `k` takes **O(k)** time.

What is the **overall time complexity** of sorting the list?

OPTIONS :

- ○ $O(n\ k)$
- ✓ ○ $O(k\ n\ log\ n)$
- ○ $O(k\ n\ log\ k)$
- ○ $O(n^2\ log\ k)$

'n' strings, each of length k

Merge Sort's Time Complexity
= O(P log P) P = len(list)

Lexicographical order ≡ arranging sequences
→ words (dictionary order)
→ numbers (digit wise order) 1, 2, 3, 10, 11, 21 ×
1, 10, 11, 2, 21, 3 (lexicographical)
→ tuples (based on first item then second item)

```
def Merge(): # O(n) ⇒ for string comparison
    # logic here                O(k)
                          So overall ⇒ O(n·k)

def MergeSort(l):
    if len(l) ≤ 1:
        return l
    mid = ....
    left = MergeSort(l[0:mid])
    right = MergeSort(l[mid:])
```

So, Total T.C. = O(k·n log n)

⇒ R(n) = 2·R(n/2) + O(n·k)

```
left = MergeSort(l[0:mid])
right = MergeSort(l[mid:])
return merge(left, right)
```

Consider the **Quicksort algorithm** where the **first element of the list** is always chosen as the **pivot** for partitioning.

Examine the following statements:

1. The best-case performance occurs when the pivot always divides the list into two equal halves. ✓
2. The best-case performance occurs when the input list is already sorted in ascending order. ✗ worst case
3. The worst-case performance occurs when the pivot always divides the list into two equal halves. ✗ best case

Which of the above statements is/are **correct**?

OPTIONS:

- ✓ Only Statement 1
- Only Statement 2
- Only Statement 3
- Statements 1 and 2
- Statements 2 and 3

Assume `Stack` is a class implementing the **stack** data structure that supports the standard operations:

- `push(x)` : inserts an element `x` onto the stack
- `pop()` : removes and returns the top element of the stack
- `isempty()` : returns `True` if the stack is empty, and `False` otherwise

Now consider the function `fun` given below:

```
1  def fun(S):
2      if not S.isempty():
3          x = S.pop()
4          fun(S)
5          S.push(x * 2)
6          S.push(x)
7
8  S = Stack()
9  L = [1, 2, 3]
10 for v in L:
11     S.push(v)
12 fun(S)
```

stack = [3, 2, 1]

fun(3) x=3 → lines 5, line 6 → 3, 3x2, 2, 2x2, 1, 1x2

fun(2) x=2 → lines 5, line 6 → 2, 2x2, 1, 1x2

fun(1) x=1 → lines 5, line 6 → 1, 1x2

fun() Do nothing

What will be the final contents of the stack **S** at the end of program (from **bottom to top**)?

OPTIONS:

- [1,2,3,2,4,6]
- [3,2,1,6,4,2]
- ✓ [2,1,4,2,6,3]
- [6,4,2,3,2,1]

[3, 6, 2, 4, 1, 2]

```
1  class Node:
2      def __init__(self,data):
3          self.data = data
4          self.next = None
```

A queue is implemented using a linked list. Each node of the linked list is an object of class `Node`. The queue has a `head` pointer that points to the first node of the linked list, and a `rear` pointer that points to the last node of the linked list.

Let `n` denote the number of nodes in the queue. Let `enqueue()` be implemented by inserting a new node at the `rear`, and `dequeue()` be implemented by deletion of a node from the `head`.

Which one of the following is the time complexity of the most time-efficient implementation of `enqueue()` and `dequeue()`, respectively, for this data structure?

OPTIONS:

- $O(1), O(1)$
- $O(1), O(n)$
- $O(n), O(1)$
- $O(n), O(n)$

```
new_node = Node(data)
rear.next = new_node
```
enqueue(data)

```
temp = head
head = temp.next
```
dequeue()

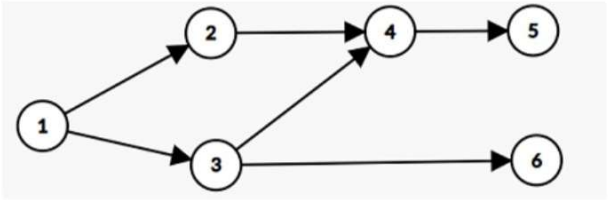
Note: Without rear pointer, enqueue() takes O(n)

Select the most appropriate data structure for the following applications.

Application	Data Structure
1. Undo operation in a text editor	a. Queue
2. Managing print jobs in a printer	b. Stack
3. Storing and accessing book records by ID	c. Graph
4. Finding the shortest path between two cities	d. Hash Table

1-b, 2-a, 3-d, 4-c

Consider the following graph.



Which among the following is/are **not** a topological ordering of the given graph ?

OPTIONS :

- [ ] [1, 2, 3, 4, 6, 5]
- [x] [1, 3, 4, 2, 5, 6]
- [ ] [1, 3, 2, 4, 6, 5]
- [ ] [1, 2, 3, 4, 5, 6]
- [x] [1, 2, 4, 3, 5, 6]

1, (2,3), (6), 4, 5

Consider a **connected, undirected graph G** with **n** vertices. A **Breadth-First Search (BFS)** traversal is performed starting from a source vertex **s**. Let **d(s, v)** denote the shortest distance (in terms of number of edges) from **s** to vertex **v**.

Which of the following statements is **true** about the BFS traversal?

OPTIONS :

- [x] All vertices at distance **k** from **s** are visited before any vertex at distance **k + 1** from **s**.
- [x] Once a vertex is visited, it is never visited again during the traversal.
- [ ] BFS may fail to visit some vertices, even if the graph is connected.
- [ ] The time complexity of BFS using an adjacency list representation is [redacted]

$O(n^2)$

adjacency list

$O(V+E)$

$O(n + \text{no. of edges})$

```python
def g(n):
    count = 0
    for i in range(1, n + 1):
        if n % i == 0 and i % 2 == 0:
            count = count + 1
    return count
```

What is `g(48) - g(36)`, given the definition of `g` above?

2,4,6,8,12,16,24,48

g(48) = 8

2,4,6,12,18,36

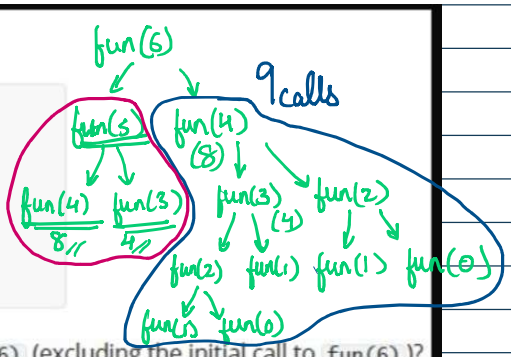
g(36) = 6

= 2

Consider the code given below:

```python
def fun(n):
    if n <= 1:
        return 0
    return fun(n - 1) + fun(n - 2)

fun(6)
```



3 + 8 + 4

→ 24 excl. fun(6)

How many recursive calls are made to evaluate `fun(6)` (excluding the initial call to `fun(6)`)?

Linear probing is a collision resolution technique used in open addressing hash tables. When a collision occurs at a hashed index, linear probing searches for the next available slot by **incrementing the index by 1** (modulo the table size) until a free slot is found.

Consider a **hash table with 8 buckets**, indexed from **0 to 7**, using the hash function:

`hash(key) = key mod 8`

The table uses **linear probing** to resolve collisions.

0	1	2	3	4	5	6	7
	25	10	3	18	33	42	

If the following key values are inserted in the given order:

`10, 3, 18, 25, 33`

At which **index** will the key value **42** be inserted?

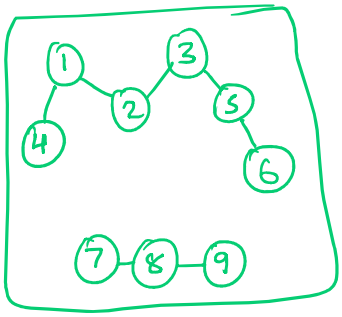
6

An undirected connected graph `G` has 20 vertices. The maximum number of edges in `G` is 190

$\frac{20 \times 19}{2}$

Given below is the adjacency list of a undirected graph:

```python
Alist = {
    1: [2,4],
    2: [1,3],
    3: [2,5],
    4: [1],
    5: [3,6],
    6: [5],
    7: [8],
    8: [7,9],
    9: [8]
}
```



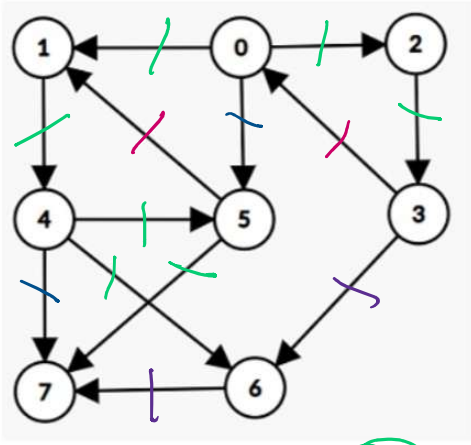
2 components

x = 2

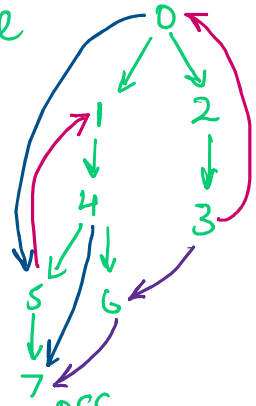
x + y = 4

If x is the degree of the node 5 in the graph and y is the number of components in the graph, then what is the value of $x + y$?

You are given the following **directed graph**:



Tree



Tree edges

Forward edges

Backward edge

Cross edges

Let **Depth-First Search (DFS)** start from vertex 0. How many cross edges will be present in the DFS tree?

Note:- If more than one adjacent vertex is available, DFS always chooses the vertex with the smaller label value first.

Refer old TA-session recording or Notes

Date of recording : 24/10/2026